

Scalable Updatable Secure Storage from Linear-Size Homomorphic Vector Commitments

Long Chen Shuangyu He

Institute of Software, Chinese Academy of Sciences, Beijing 100190

chenlong@iscas.ac.cn, heshuangyu@iscas.ac.cn

Abstract

Updatable encryption (UE) rotates ciphertext keys from a short token. Updatable secure storage (USS) adds version integrity: a client outsources a dynamic encrypted database, keeps one short stub, and detects a malicious server that serves a stale file—even after the current secret leaks. The only published USS construction (Chen–Guo–Li–Tang, ASIACRYPT 2023) is theoretical in practice: its Catalano–Fiore HVC carries an $O(n^2)$ CRS (tens of megabytes already at $n = 2^{10}$), and it left the large-file DEM unmeasured. This paper makes USS practical by removing those two bottlenecks. First, KHVC is a KZG-based HVC with an $O(n)$ SRS, tight q -SDH position binding, and statistical position hiding; at $n = 1024$ ParGen drops from 70.2s to 0.38s and operational parameters from 50.5 MB to 96.3 KB. Second, CGLT23 already allows any IND-CPA UE as the DEM (integrity is OF-PTXT via the stub), so we instantiate a measured random-oracle Ring-LWR key-homomorphic PRF with no MAC, sized for 127 correct updates, and cache the public a_b NTTs so KeyUp does not rebuild them. FileUp openings use a one-hot closed form instead of a dense FK20; openings for KeyUp still use precomputed $\{W_j\}$. We implement the scheme and the 2023 HVC baseline in Rust (BLS12-381) and measure real costs, including 1 GB files.¹ On an Apple M5 Pro, at 1 MB the DEM alone runs at 32.6 MB/s with a $2\times$ ciphertext and a 4096-byte token. End-to-end KeyUp of four 1 GB files is 254s and a 4240-byte token, against tens of minutes to hours if SHINE0 or RISE is applied to the same bytes. At $n = 1024$, a one-file FileUp opening costs 194ms on KHVC versus about 62ms for Catalano–Fiore.

Keywords: vector commitment, KZG, updatable encryption, updatable secure storage, key rotation.

1 Introduction

Cloud storage is encrypted, and industry standards (PCI DSS, AWS/GCP CMEK) demand periodic key rotation. Updatable encryption (UE) [4, 5, 7, 6] lets a server re-key a ciphertext from a short token, without learning the plaintext. That is not enough for a dynamic database: after a file update, a malicious server can still return an old ciphertext that decrypts under the current key. Chen, Guo, Li, and Tang [10] introduced updatable secure storage (USS) to add *version integrity*: the client keeps one short stub, FileUp and KeyUp communication does not grow with the number of files, and a cheating server is caught even if the current secret has leaked. Their architecture is KEM+DEM: an HVC commits to file hashes, RISE wraps the openings, and the DEM is any ciphertext-independent UE that is only IND-CPA—OF-PTXT recomputes the file commitment after decryption, so the DEM needs no MAC.

That blueprint was never turned into a working large-file system. Two bottlenecks blocked it. The 2023 HVC is Catalano–Fiore [2]: the public parameters contain every pairwise product $g^{z_i z_j}$,

¹Artifact: <https://github.com/longchenHefei/USS-Optimized-KZG>. Reproduce with `cargo test --release -p uss-khvc`, `cargo run --release --bin fairbench`, `cargo run --release --bin rlwrbench`, and the CSVs in `experiments/results/`.

i.e. an $O(n^2)$ CRS. At $n = 1024$ this is already 50 MB in our implementation; at $n = 10^4$ it is several gigabytes, and ParGen alone takes about a minute at $n = 1024$. Separately, wrapping file bytes in RISE, SHINE0, or an elliptic-curve KH-PRF is far too slow for gigabyte payloads (Section 7), AES-GCM has no update algorithm, and nested AES forces decryption to walk every epoch. NIST limits the originator-usage period of a symmetric data-encryption key to less than two years [9, Table 1]; a file kept for years to a few decades is therefore updated tens of times. We take **128 updates** as the engineering bound: it covers annual rotation for more than a century and quarterly rotation for about thirty years, matching the nested-UE knob of [7].

This paper removes both bottlenecks and measures the result.

Contributions.

1. **KHVC (practical HVC).** A KZG [3] HVC with linear SRS, $\text{aux} = \rho \in \mathbb{F}_p$ (succinct, as required by [10]), tight q -SDH position binding (no $1/n$ loss), statistical position hiding, and public rerandomization. One-hot FileUp openings use a closed form in the precomputed vanishing openings $\{W_j\}$, so a single-slot update does not run a dense FK20.
2. **CPA Ring-LWR DEM.** We follow CGLT23’s allowance that the DEM may be any IND-CPA UE: integrity stays with the stub. We instantiate a random-oracle Ring-LWR key-homomorphic PRF [7], sized for 127 correct updates (8-bit noise pad), store public a_b NTTs with each file so KeyUp does not rebuild them, and drop the redundant embedding of h_i inside the body ciphertext that CGLT23 wrote into the DEM. KeyUp sends the RISE token and $\Delta = s' - s$; the server rewrites every body.
3. **Implementation and measurements.** A Rust crate with KHVC, Catalano–Fiore HVC, RISE, SHINE0, ReCrypt-style KH-PRF, nested AES-GCM, the Ring-LWR DEM, and end-to-end USS on BLS12-381. We compare the 2023 CF-HVC backend with KHVC at the same n , and we compare payload UE, including the 128-update Ring-LWR DEM, on the same machine.

We do not claim that “hidden KZG” is new [3, 14], nor that CPA-only DEM integrity is a new USS theorem—that is already in [10]. The cryptographic novelty is an HVC that meets the USS interface with a linear SRS; the systems novelty is a working USS for large files, with the DEM and FileUp/KeyUp engineering that the 2023 paper left open.

2 Preliminaries

Let $\mathbb{F}_p$ be the scalar field of BLS12-381 (the group order). We write $[a]_1 = ag_1$ and $[a]_2 = ag_2$. The KHVC blinding scalar is $\rho \in \mathbb{F}_p$; RISE’s ephemeral scalar is $r \in \mathbb{F}_p$; a file-commitment nonce is $\nu_i \in \{0, 1\}^{256}$. These three objects are never the same symbol.

Pairings. A Type-3 pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ on BLS12-381.

Assumption 1 (SXDH). *DDH is hard in $\mathbb{G}_1$ and in $\mathbb{G}_2$.*

Assumption 2 (q -SDH [1, 3]). *Given $([1]_1, [x]_1, \dots, [x^q]_1, [1]_2, [x]_2)$, a PPT adversary outputs $(c, [1/(x+c)]_1)$ with $c \neq -x$ only with negligible probability. The reduction uses this tuple as the KHVC SRS of degree $q = n$.*

KZG [3]. $\text{Setup}(1^\lambda, d)$ samples τ and publishes $([1]_1, \dots, [\tau^d]_1, [1]_2, [\tau]_2)$. $\text{Com}(f) = [f(\tau)]_1$. An opening of f at z is $[q(\tau)]_1$ for $q(X) = (f(X) - f(z))/(X - z)$. Evaluation binding follows from q -SDH. Commitments and openings are additively homomorphic. Verification uses only $[1]_2$ and $[\tau]_2$; it does not need $[\tau^d]_2$.

Definition 1 (HVC [10]). A homomorphic vector commitment consists of $\text{Setup}(1^\lambda, n)$, $\text{Com}(\text{pp}, \mathbf{m}) \rightarrow (C, \text{aux})$, $\text{Open}(\text{pp}, \mathbf{m}, \text{aux}, i) \rightarrow \Lambda_i$, $\text{Ver}(\text{pp}, C, i, m, \Lambda_i)$, $\text{ComHom}(C, C')$, $\text{OpenHom}(\Lambda_i, \Lambda'_i)$. The auxiliary string aux is independent of n . Position binding: no PPT adversary outputs $C, i, m \neq m', \Lambda, \Lambda'$ that both verify. Position hiding: given C and $\{\Lambda_j\}_{j \neq i}$ as group elements, the slot- i message is statistically hidden (Theorem 2). Homomorphism: $C \oplus C'$ and $\Lambda_i \otimes \Lambda'_i$ open to the coordinate-wise sum.

RISE [4]. ElGamal-style UE in a prime-order group: $\text{Enc}_k(M) = (rY, rG + M)$ with $Y = kG$, token $\Delta = k'/k$, update re-randomizes and multiplies C_1 by Δ . Lehmann–Tackmann prove IND-ENC and rand-IND-UPD from DDH in that group. We instantiate the group as $\mathbb{G}_1$; SXDH is the Type-3 stand-in for that DDH assumption. RISE is additively homomorphic on the message group, which is what USS needs for encrypted HVC openings. It is *not* a large-file cipher: the message is a group element.

USS [10]. A database of n files. **Store** produces a repository of ciphertexts and a stub C . **Rev** returns file i or $\perp$. **FileUp** replaces file i ; **KeyUp** rotates keys. Security games, inherited verbatim from [10]: IND-DD-UP, IND-FileUp-CPA, IND-REENC-CPA, and OF-PTXT (integrity against a cheating server who may know the secret key). Section 1 lists the hypotheses under which we instantiate those games; Section 5 gives the hops.

Remark 1 (Instantiation hypotheses). The rest of the paper uses: (i) Type-3 BLS12-381 and SXDH; (ii) q -SDH with $q = n$ for KHVC binding; (iii) a trusted KZG SRS (knowledge of τ is forbidden to all parties after setup); (iv) SHA-256 modelled as a random oracle, both as COM and as the ring elements $a_b = H(\text{tweak} \| b)$; (v) file nonces ν_i of 256 bits of entropy; (vi) RISE in $\mathbb{G}_1$, IND-ENC and rand-IND-UPD under SXDH [4]; (vii) decisional Ring-LWR in dimension $N = 1024$, $R_q = \mathbb{Z}_q[X]/(X^N + 1)$, $q = 998244353$, rounding modulus $p = 2^{16}$, and a uniform secret in R_q . The DEM is correct for at most 127 updates: each update adds at most 1 to the rounding error, and eight noise bits absorb an error of absolute value at most 127.

3 KHVC

Let $\Omega = \{\omega^0, \dots, \omega^{n-1}\}$ be the n -th roots of unity and $Z(X) = X^n - 1$.

Setup. Publish the degree- n KZG SRS $[1]_1, \dots, [\tau^n]_1$ ($n + 1$ compressed $\mathbb{G}_1$ elements) and $[1]_2, [\tau]_2$. No $[\tau^n]_2$ is required: the pairing check below only uses $[\tau]_2 - [\omega^i]_2$. The hiding shift $[Z(\tau)]_1 = [\tau^n]_1 - [1]_1$ is free from the SRS. For **KeyUp**, precompute the table $W_j = [Z(\tau)/(\tau - \omega^j)]_1$ by one $\mathbb{G}_1$ FFT of the SRS (data-independent; n extra $\mathbb{G}_1$ elements, 48 KB at $n = 1024$). Table 1 reports two sizes. The verification SRS counted by `crs.bytes()` is $(n + 1) \cdot 48 + 2 \cdot 96 + 48$ bytes: 48.3 KB at $n = 1024$, which is $50.5 \text{ MB} / 48.3 \text{ KB} \approx 1070\times$ smaller than Catalano–Fiore. That counter excludes $\{W_j\}$ and includes one redundant 48-byte copy of $[Z(\tau)]_1 = [\tau^n]_1 - [1]_1$. Operational parameters are the same string plus the n compressed openings W_j , hence $48.3 + 48 = 96.3$ KB. The raw ratio against the measured Catalano–Fiore CRS (50,528,400 bytes) is $50,528,400 / (49,440 + 49,152) \approx 512$.

Commit. Interpolate $f(\omega^i) = m_i$ ($\deg f < n$), sample $\rho \leftarrow \mathbb{F}_p$, set $F = f + \rho Z$ and $C = [F(\tau)]_1$. Output $(C, \text{aux} = \rho)$. The client keeps ρ ; the server never receives it. Coefficient form: F has length $n + 1$, namely $(f_0 - \rho, f_1, \dots, f_{n-1}, \rho)$.

Open / Ver. $\Lambda_i = [(F(X) - m_i)/(X - \omega^i)](\tau)$; pairing check $e(C - [m]_1, [1]_2) = e(\Lambda_i, [\tau]_2 - [\omega^i]_2)$. The ρZ term does not change evaluations on Ω ($Z(\omega^i) = 0$), so no extra $\mathbb{G}_2$ SRS element appears.

Hom / ReRand. $C + C'$ and $\Lambda_i + \Lambda'_i$. $\text{ReRand}(C) = C + \rho'[Z(\tau)]_1$, which does not change any $F(\omega^i)$. If τ were known, rerandomization would be meaningless; trusted setup is essential.

Theorem 1 (Binding). *If q -SDH holds then KHVC is position binding. An adversary with advantage ε yields a q -SDH solution with advantage $\varepsilon - O(1/p)$.*

Proof. Two accepting openings of the same C at the same ω^i to $m \neq m'$ satisfy

$$e(C - [m]_1, [1]_2) = e(\Lambda, [\tau]_2 - [\omega^i]_2), \quad e(C - [m']_1, [1]_2) = e(\Lambda', [\tau]_2 - [\omega^i]_2).$$

Dividing and scaling by $(m' - m)^{-1}$ produces $[1/(\tau - \omega^i)]_1$. The reduction is given the q -SDH tuple $([1]_1, \dots, [x^n]_1, [1]_2, [x]_2)$, publishes it as the KHVC SRS with x playing the role of τ , and returns $(c, [1/(x + c)]_1)$ for $c = -\omega^i$. It does not guess i . The $O(1/p)$ loss is the probability that $m' - m = 0$ after a wrap in $\mathbb{F}_p$, which the winning condition $m \neq m'$ already excludes up to that encoding. $\square$

Theorem 2 (Statistical hiding). *Let the adversary see C and $\{\Lambda_j\}_{j \neq i}$ as $\mathbb{G}_1$ elements. Except with probability at most n/p over setup (the event $\tau \in \Omega$), the two position-hiding worlds are identical, and $\rho_0 \mapsto \rho_1$ below is a translation. An unbounded distinguisher therefore has advantage at most n/p .*

Proof. The adversary names a slot i and two vectors that agree off i . Write $\Delta m = m_{i,1} - m_{i,0}$ and let L_i be the Lagrange basis polynomial for ω^i on Ω , so the interpolated polynomials satisfy $f_1 = f_0 + \Delta m L_i$. If $\tau \notin \Omega$ then $Z(\tau) \neq 0$. Given the blinding ρ_0 of the first world, set

$$\rho_1 = \rho_0 - \Delta m \cdot \frac{L_i(\tau)}{Z(\tau)}.$$

Then

$$F_1(\tau) = f_1(\tau) + \rho_1 Z(\tau) = f_0(\tau) + \Delta m L_i(\tau) + \left(\rho_0 - \Delta m \frac{L_i(\tau)}{Z(\tau)} \right) Z(\tau) = F_0(\tau).$$

Hence C is the same group element in both worlds. For every $j \neq i$ the opening is the group element $\Lambda_j = [(F(\tau) - m_j)/(\tau - \omega^j)]_1$. It depends on F only through the scalar $F(\tau)$, and m_j is the same in both worlds, so Λ_j is the same group element. The map $\rho_0 \mapsto \rho_1$ is a translation of $\mathbb{F}_p$, so a uniform ρ_0 yields a uniform ρ_1 . The equality is numerical at the setup point τ . It is not a polynomial identity, and the adversary is not given the coefficient vector of F . $\square$

Homomorphism is the KZG homomorphism of F and of the quotients.

Lemma 1 (All-openings decomposition). *Let $F = f + \rho Z$ with $\deg f < n$ and $m_i = F(\omega^i) = f(\omega^i)$. Then*

$$\frac{F(X) - m_i}{X - \omega^i} = \frac{f(X) - m_i}{X - \omega^i} + \rho \cdot \frac{Z(X)}{X - \omega^i},$$

because $Z(\omega^i) = 0$. The first summand has degree at most $n - 2$; the second has degree $n - 1$. Equivalently, the coefficient vector of F is $(f_0 - \rho, f_1, \dots, f_{n-1}, \rho)$, and FK20 [13] is run on the length- n prefix $f - \rho$ (a degree- $< n$ polynomial). Hence $\Lambda_i = \pi_i^{\text{FK20}}(f - \rho) + \rho \cdot W_i$ with $W_i = [Z(\tau)/(\tau - \omega^i)]_1$. The table $\{W_j\}$ is computed once from the SRS by a single $\mathbb{G}_1$ FFT; query-time KeyUp does not run n MSMs. A naïve synthetic division plus MSM is $O(n^2)$ group operations; FK20 is a Toeplitz embedding, a length- $2n$ $\mathbb{G}_1$ FFT, and a length- n FFT, i.e. $O(n \log n)$ group additions. KeyUp opens the zero vector, so only the second term remains: n scalar multiplications by the precomputed W_j .

Lemma 2 (One-hot FileUp openings). *Let $\Delta \in \mathbb{F}_p$, let $i \in \{0, \dots, n-1\}$, and let $f = \Delta L_i$ where L_i is the Lagrange basis polynomial for ω^i on Ω , so $L_i(X) = (\omega^i/n) Z(X)/(X - \omega^i)$. Write $F = f + \rho Z$ and $W_j = [Z(\tau)/(\tau - \omega^j)]_1$. Then for every $j \neq i$,*

$$\Lambda_j = \Delta \cdot \frac{\omega^i}{n} \cdot \frac{W_i - W_j}{\omega^i - \omega^j} + \rho \cdot W_j.$$

*The opening at slot i is the ordinary KZG quotient of $F - \Delta$ at ω^i (one synthetic division and one MSM of length n). Both agree with **Open** on the one-hot vector; the dense FK20 path is unnecessary when a FileUp touches a single slot.*

4 USS instantiation

We follow the VC-then-UE template of [10, §5]. Let $\text{COM}(m_i; \nu_i) = \text{SHA-256}(m_i \parallel \nu_i)$ mapped into $\mathbb{F}_p$ by interpreting the digest as a little-endian integer. In the ROM this is hiding (the nonce has 256 bits) and binding (finding $(m, \nu) \neq (m', \nu')$ with the same digest is a RO collision). Unlike [10], the DEM ciphertext is $\nu_i \parallel m_i$ only: h_i is recomputed on **Rev** and is not stored inside the body.

DEM (ROM Ring-LWR). The epoch key is one uniform secret $s \in R_q$, $R_q = \mathbb{Z}_q[X]/(X^N + 1)$. A file is stored under a fresh public tweak. For each block b , $a_b = H(\text{tweak} \parallel b)$ and

$$F_s(b) = \left\lfloor \frac{p}{q} (a_b \cdot s) \right\rfloor \in R_p.$$

The ciphertext is $\text{encode}(\nu_i \parallel m_i) + F_s(b)$, one ring element per 1024 payload bytes, with eight message bits and eight noise bits in each coefficient. The KeyUp token is $\Delta = s' - s \in R_q$ (4096 bytes). The server adds $F_\Delta(b)$ to every block. In $\mathbb{Z}_p$ this is a fresh encryption under s' plus a rounding error of infinity norm at most 1 per update, hence at most 127 after 127 updates, which the pad absorbs. Each file record stores the twisted NTT of every public a_b alongside the tweak (about 8 KB per block, rebuildable from the tweak, not a secret). KeyUp and Rev reuse that cache; they do not re-hash and re-NTT a_b on every block. Openings are still wrapped by RISE in $\mathbb{G}_1$. There is no per-file AES seed: body confidentiality is this DEM, not a re-wrap of a static key. We claim CPA only, as already permitted for the USS DEM in [10]. OF-PTXT recomputes COM after decryption, so a mauled body that decodes to a different file fails verification, and the DEM carries no MAC.

ParGen($1^\lambda, n$). Run **KHVC.Setup**($1^\lambda, n$). Output **pp**.

KeyGen(**pp**). Sample a RISE key pair $(k, Y = kg_1)$ and a uniform DEM secret $s \in R_q$. The USS secret is (k, s) , not ρ .

Store(**pp**, $k, s, (m_1, \dots, m_n)$). For each i : sample ν_i and a tweak; encrypt $\nu_i \parallel m_i$ under F_s as above and cache the a_b NTTs; RISE-encrypt the KHVC opening Λ_i . Let $h_i = \text{COM}(m_i; \nu_i)$. The client samples a fresh ρ , sets $(C, \rho) \leftarrow \text{KHVC.Com}(h_1, \dots, h_n)$, and sends the repository (Ring-LWR bodies, tweaks, cached a_b , RISE openings) and stub C to the server. The client may discard ρ after Store: later FileUp/KeyUp sample a new ρ_δ or ρ_0 . The server never receives $\text{commit-}\rho$.

Rev(**pp**, k, s, repo, i). RISE-decrypt the opening; subtract $F_s(b)$ using the cached a_b NTT; decode $\nu_i \parallel m_i$; recompute h_i ; **KHVC.Ver** the stub at i ; output m_i or $\perp$.

FileUp(pp, k , repo, i , m'_i). The client downloads file i , recovers (m_i, ν_i) , samples ν'_i and $h'_i = \text{COM}(m'_i; \nu'_i)$, and builds a one-hot delta with nonzero slot $h'_i - h_i$. It samples ρ_δ , commits the delta, and sends the delta commitment together with a fresh Ring-LWR ciphertext of the new file under the current s and a fresh tweak (with a fresh a_b cache). The server opens the delta at all n points by Lemma 2 and homomorphically adds each RISE-wrapped opening. Communication is $O(|m'_i|)$ plus $O(1)$ group elements, independent of n and of other files. Dense all-open (FK20) remains available when many slots change at once; the single-file path measured below uses the one-hot form.

KeyUp(pp, (k, s) , (k', s') , repo). The client sends a RISE token $(\Delta_R = k'/k, Y' = k'g_1)$, a fresh hiding nonce $\rho_0 \in \mathbb{F}_p$, and the DEM token $\Delta = s' - s \in R_q$. The implementation returns 4240 bytes.² The server RISE-updates every opening, adds $\text{RISE.Enc}_{k'}(\rho_0 \cdot W_j)$ to each opening, adds $[\rho_0 Z(\tau)]_1$ to the stub, and adds $F_\Delta(b)$ to every body block using the cached a_b NTTs. Bodies are rewritten eagerly so that IND-REENC-CPA matches [10] without a lazy-update game. Table 8 times this full KeyUp.

Lazy body KeyUp. One could defer the body scan until Rev or FileUp and only rotate the RISE header immediately. That would cut KeyUp wall-clock on cold archives, but the intermediate repository is then a body under an old secret plus a public chain of Δ 's, and IND-REENC-CPA would need a lazy restatement. We leave that variant as future work and measure the eager scan.

Why Type-3 RISE. Openings lie in $\mathbb{G}_1$, so RISE must be instantiated there (SXDH). The 2023 paper used a symmetric pairing group, where DDH is easy; Type-3 is the sound instantiation.

Full payload update is the Ring-LWR DEM above: ciphertext-independent, CPA, and correct for 127 updates at this pad. SHINE/OCBSHINE [6] and elliptic-curve ReCrypt [5] are the same kind of DEM and are too slow on file bytes (Section 7). Theorem 3 claims body IND-CPA and body update for this DEM, not for AES-GCM.

5 Security

Theorem 3. *Under Remark 1, and writing Adv for the usual USS advantages of [10]:*

1. *KHVC is statistically position hiding (Theorem 2) and tightly q -SDH position binding.*
2. *Header and stub IND-DD-UP, IND-FileUp-CPA, and IND-REENC-CPA follow the hybrids of [10, Thms. 3–5], with RISE in $\mathbb{G}_1$ under SXDH and with every pairing check in Type-3.*
3. *OF-PTXT follows from KHVC binding and a random-oracle collision of COM (Lemma 3), even if the current RISE key and every KeyUp nonce ρ_0 are leaked.*
4. *A body is IND-CPA under decisional Ring-LWR in the random oracle, and so is a body that has been updated at most 127 times. The updated ciphertext differs from a fresh encryption under the new secret by a rounding error of infinity norm at most the number of updates. The DEM is not authenticated encryption.*

The hops below are those of [10, Thms. 3–6] under the substitutions written here. This paper does not reprove Chen–Guo–Li–Tang.

²Header fields: $\Delta_R \in \mathbb{F}_p$ (32 B), $Y' \in \mathbb{G}_1$ (48 B), ρ_0 (32 B), plus a 32-byte counter word (144 B). DEM token: N coefficients of $\mathbb{Z}_q$ (4096 B). Total $144 + 4096 = 4240$.

DEM in the random oracle. Let H be the random oracle used as COM and as $a_b = H(\text{tweak}||b)$. Replacing a body by uniform elements of R_p is decisional Ring-LWR for the epoch secret: each block is a rounded product $a_b \cdot s$, and a_b is a fresh oracle output. The same hop applies after KeyUp. The server is given $\Delta = s' - s$. Since s' is uniform, Δ is uniform and independent of s , and the updated block is $F_{s'}(b) + \text{encode}(\nu||m) + e$ with $\|e\|_\infty \leq t$ after t updates. For $t \leq 127$ the pad removes e on decryption, and e does not depend on the message. Distinguishing the updated body from uniform is Ring-LWR under s' . Seeing both the old and the new body does not help beyond computing F_Δ , which the server can already compute from Δ .

IND-DD-UP. Follow [10, Thm. 3]. H0 is the real game. H1 replaces the challenge file's COM output by a uniform $\mathbb{F}_p$ element; the hop fails only on a random-oracle query to the challenge nonce. H2 replaces the challenge openings by RISE encryptions of uniform $\mathbb{G}_1$ elements (RISE IND-ENC under SXDH). H3 replaces the Ring-LWR body by uniform R_p elements (the DEM hop above). H4 invokes Theorem 2 on the stub and on the openings of every slot except the challenge slot. The Type-3 change is local: every pairing check is $e(C - [h]_1, [1]_2) = e(\Lambda, [\tau]_2 - [\omega]_2)$ with $\Lambda \in \mathbb{G}_1$. Nothing is extracted from $\mathbb{G}_2$.

IND-FileUp-CPA. Follow [10, Thm. 4]. The FileUp delta is one-hot. H1 hides the new COM value in the random oracle. H2 invokes RISE IND-ENC on the one-hot openings of the delta (Lemma 2), and Ring-LWR on the fresh body under the current s (a new tweak makes a_b a fresh oracle string). H3 invokes Theorem 2 on the updated stub. The same Type-3 pairing check replaces any symmetric-group extraction step in [10].

IND-REENC-CPA. Follow [10, Thm. 5] for headers and the stub. That hop is RISE rand-IND-UPD in $\mathbb{G}_1$, together with the KHVC rerandomization $\rho_0 W_j$. Bodies are inside the claim: the DEM hop above, under the new secret s' , for at most 127 updates.

Lemma 3 (OF-PTXT under key leakage). *OF-PTXT holds by KHVC binding and COM binding in the ROM, even if the current RISE key, the DEM secret, and every KeyUp nonce ρ_0 are leaked. Updating the body is not what the integrity argument uses.*

Proof. The adversary of [10, Thm. 6] may hold the RISE key, the DEM secret, and every KeyUp nonce ρ_0 , and may drive FileUp and KeyUp. A win is a repository that verifies at slot i for a plaintext that is not the latest honest file at i .

After an honest FileUp the stub is $C' = C \oplus C_\delta$ and the honest opening at i verifies for h'_i . A stale plaintext whose commitment is $h_i \neq h'_i$, together with an opening Λ^* that verifies for C' , is two accepting openings of one slot of C' to two messages. That is a KHVC binding break, and the reduction in the binding theorem turns it into q -SDH. If the stale bytes hash to h'_i , the same transcript is a random-oracle collision for COM. The reduction does not need the RISE key.

KeyUp adds $[\rho_0 Z(\tau)]_1$ to the stub and $\rho_0 W_j$ to each opening. Both are multiples of Z , so every evaluation on Ω is unchanged. Reusing a pre-FileUp opening, which verifies the old stub for h_i , does not verify the updated stub for h_i : the updated stub opens to h'_i . A second opening to h_i is again binding. Knowledge of ρ_δ lets the adversary recompute C_δ . It does not let the adversary replace the stub the client stores. $\square$

6 Implementation

A single Rust crate (`uss-khvc`) on arkworks 0.5 (BLS12-381), `aes-gcm` (AES baselines only), and `curve25519-dalek` 4.1. File bodies in `uss.rs` are the Ring-LWR DEM with cached a_b NTTs; KeyUp adds F_Δ to every block without rebuilding a_b . FileUp openings use `open_one_hot`; dense

FK20 remains for microbenchmarks of $k = n$. The USS timings in Table 8 are one-shot measurements from `experiments/results/uss.csv` on this build. KHVC uses `Radix2EvaluationDomain` IFFTs and an explicit $\mathbb{G}_1$ Cooley–Tukey FFT for FK20. Catalano–Fiore HVC stores $h_i \in \mathbb{G}_1$, $\hat{h}_i \in \mathbb{G}_2$, and off-diagonal $h_{i,j} \in \mathbb{G}_1$ (Type-3 verification $e(C - mh_i, \hat{h}_i) = e(\Lambda_i, g_2)$). UE baselines live in the same process so Enc/Dec/Upd numbers are comparable. The crate now selects the naïve all-open for $n < 1024$ and FK20 otherwise. Table 1 and the USS rows are one-shot timings from a CSV that predates that switch. The switch does not change the $n \geq 1024$ cells, and the small- n USS rows were not remeasured under it.

We also looked for public UE code. SHINE has a Python/C prototype [17]; ReCrypt has an abandoned Rust prototype [18]; nested AES and RLWE KH-PRF have C code [19]. IronCore `recrypt-rs` is proxy re-encryption, not UE. RISE has no public implementation; we wrote it. Primary tables use the same Rust stack; we did not mix MIRACL C timings into the main table.

7 Evaluation

Machine. Apple M5 Pro, 24 GB RAM, 15 threads, macOS 26.6.2, rustc 1.97.1, release with LTO. HVC and USS cells are one-shot timings from `experiments/results/all.csv`. Tables 2–3 are medians from `fairbench` (`experiments/results/fair_ue.csv`): a discarded warm-up, then 21 repeats at header size and 4 KB, 5 at 64 KB, 3 at 1 MB algebraic, and 7 at 1 MB AES/KH-PRF. Table 4 is the same median accounting from `rlwrbench` (`experiments/results/rlwr_ue.csv`): warm-up discarded, then 21 repeats at 4 KB, 7 at 64 KB, and 5 at 1 MB, single-threaded. We do not time RISE or SHINE0 at 100 MB (1 MB RISE Enc is already 2.6 s). A 100 MB AES/Nested figure is taken from the historical `bench` suite; ReCrypt at 100 MB is $100\times$ the 1 MB median and is labeled extrapolation.

7.1 HVC: linear vs quadratic CRS

Table 1: KHVC vs Catalano–Fiore HVC. Times in milliseconds, one-shot. The CRS column is the verification counter (`crs_bytes`): it excludes $\{W_j\}$ and includes one redundant 48-byte $[Z(\tau)]_1$. CF-HVC is omitted for $n \geq 4096$.

n	scheme	CRS	setup	Com	Open+Ver	all-open
64	KHVC	3.3 KB	17	0.86	2.20	62 (FK20) / 36 (naive)
64	CF-HVC	204 KB	298	3.86	5.22	259
256	KHVC	12.3 KB	79	1.03	2.24	326 / 216
256	CF-HVC	3.05 MB	4475	15.5	16.8	3995
1024	KHVC	48.3 KB	379	2.36	3.45	1624 / 1694
1024	CF-HVC	50.5 MB	70249	61.8	63.0	63131
4096	KHVC	192 KB	1734	6.39	7.07	7707
16384	KHVC	768 KB	8200	17.9	18.0	36495

The verification-SRS ratio at $n = 1024$ is $50.5 \text{ MB} / 48.3 \text{ KB} \approx 1070$. Adding the KeyUp table gives operational parameters of 96.3 KB, and $50,528,400 / (49,440 + 49,152) \approx 512$. A Catalano–Fiore CRS at $n = 2^{14}$ would be $(16384/1024)^2 \cdot 50.5 = 12.8 \text{ GB}$;³ the KHVC verification SRS is 768 KB and the operational parameters are about 1.5 MB. Table 1 shows FK20 losing to the naïve all-open for $n < 1024$. Use the naïve path there. The published CSV predates the switch in the crate; it does not change these $n \geq 1024$ cells. KeyUp with precomputed Z -openings is 63 ms at $n = 1024$ and 1.10 s at $n = 2^{14}$, versus a full FK20 all-open of 1.62 s and 36.5 s.

³ $(2^{14}/2^{10})^2 = 256$ and $256 \times 50.5 \text{ MB} = 12.8 \text{ GB}$.

7.2 Payload UE

A 48 B RISE Enc and a 4 KB AES Enc must not share a millisecond column: on the same 4 KB plaintext the costs are 2428 ns/B versus 4.21 ns/B ($577\times$). We therefore split header latency from same-size throughput. The ReCrypt-style row is a classical elliptic-curve KH-PRF on Ristretto (DDH), not the RLWE construction of [7].

Table 2: Header UE (one group element). Median milliseconds after warm-up. Token is compact and independent of file size. AES has no server token.

scheme	$ m $	Enc	Upd	Dec	$ token $	$ ct / m $
RISE ($\mathbb{G}_1$)	48 B	0.24	0.36	0.12	80 B	2
SHINE0 (Ristretto)	32 B	0.027	0.027	0.038	32 B	1

Table 3: Same $|m|$ payload UE. Enc/Upd/Dec in MB/s. Algebraic schemes are block-wise (RISE: 48 B $\mathbb{G}_1$; SHINE0 and ReCrypt-style KH-PRF: 32 B Ristretto). Median after warm-up. KHVC-USS uses the Ring-LWR DEM of Table 4 for bodies and Table 2 for openings; the USS KeyUp comparison is Table 7.

scheme	$ m $	Enc	Upd	Dec	Enc ns/B	$ ct / m $
RISE ($\mathbb{G}_1$)	4 KB	0.41	0.27	0.76	2428	2.02
SHINE0	4 KB	1.99	1.97	1.41	503	1.00
ReCrypt-KHPRF (EC)	4 KB	1.28	1.28	7.25	782	2.00
AES-256-GCM	4 KB	237	—*	239	4.21	1.01
Nested AES-GCM	4 KB	238	231	118 [†]	4.20	1.01
RISE ($\mathbb{G}_1$)	1 MB	0.40	0.26	0.73	2490	2.00
SHINE0	1 MB	1.93	1.92	1.37	519	1.00
ReCrypt-KHPRF (EC)	1 MB	1.29	1.30	7.52	776	2.00
AES-256-GCM	1 MB	241	—*	241	4.16	1.00
Nested AES-GCM	1 MB	247	246	125 [†]	4.05	1.00

* no compact server token; a client re-encrypt is 238 MB/s at 4 KB and 240 MB/s at 1 MB. [†] decrypt after one nested update (two AES layers).

Headers are the USS workload: sub-millisecond Enc/Upd/Dec and an $O(1)$ token (Table 2). Bulk is not. On the same 1 MB plaintext, AES-GCM encrypts at 241 MB/s (4.16 ns/B). SHINE0 is 1.93 MB/s ($125\times$ slower), the ReCrypt-style KH-PRF is 1.29 MB/s ($187\times$), and RISE is 0.40 MB/s ($600\times$). The 4 KB column is the same story (237 vs 1.99 vs 0.41 MB/s). AES Upd is not a token update: the server cannot refresh the body without the key. Nested AES matches Enc throughput but Dec is linear in the number of epochs. At the 128-update bound that linear walk is 128 AES decryptions, so nested AES is not a stable DEM either. A historical one-shot 100 MB AES Enc is 0.45 s; the corresponding ReCrypt figure is 81 s if the 1 MB median is scaled by 100 (extrapolation, not a timed run).

7.3 Ring-LWR DEM, sized for 128 updates

The DEM is the random-oracle Ring-LWR scheme recalled in [7, §6]: for each block index i , $F_s(i) = \lfloor (p/q)(a_i \cdot s) \rfloor$ in R_p with public $a_i = H(i)$, and the token is $\Delta = s' - s \in R_q$. It is one negacyclic product per block, not the standard-model tree PRF that multiplies once per input bit. Parameters are a performance point, not a lattice-estimator claim: $N = 1024$, $q = 998244353$, $p = 2^{16}$, eight message bits and eight noise bits per coefficient. Each update adds a rounding error of at most 1 per coefficient, so eight noise bits guarantee 127 updates. That is the 128-update bound up to a single rotation. Past that bound the pad overflows in the worst case, and a longer archive or a monthly rotation needs a wider pad or a client re-encryption. Ciphertext expansion

is 2. The token is N elements of $\mathbb{Z}_q$, hence 4096 bytes, independent of $|m|$ and of the number of files. If the NTT of every a_i is cached, that table is four times the plaintext (4 MB for a 1 MB file) and is not counted in Enc/Upd/Dec; building it for 1 MB took 35 ms.

Table 4: Ring-LWR KH-PRF DEM. Median MB/s after warm-up, single-threaded reference NTT. Public a_i excluded.

$ m $	Enc	Upd	Dec	ns/B	$ ct / m $
4 KB	33.4	33.8	34.4	30.0	2
64 KB	33.0	32.8	33.7	30.3	2
1 MB	32.6	32.6	32.9	30.7	2

On 64 blocks the largest absolute rounding error was 42 after 64 updates and 78 after 128 updates, both inside the pad ($|e| < 128$). On 2 blocks it was 144 after 255 updates, which the pad does not survive. Four 1 GB bodies at the 1 MB update median of Table 4 would be about 126 s of pure DEM work when a_b is cached. The end-to-end USS KeyUp of Table 8 stores those NTTs with each file and measures 254 s for the same four files, with a 4240-byte token (previously 404 s when a_b was rebuilt from the tweak on every block). The pure DEM estimate of 126 s assumes the 1 MB Upd median and no memory pressure; at 4×1 GB the a_b cache is about $8 \times$ the plaintext and the measured wall-clock sits between that estimate and the old rebuild path. The server rewrites a $2 \times$ ciphertext locally. This is the payload rotation the 128-update policy actually costs. It is still about an order of magnitude faster than SHINE0 or RISE on the same bytes, and AES-GCM cannot update.

Concrete cost of these Ring-LWR parameters. We estimated the DEM as unstructured LWE with the lattice-estimator [8], commit 53da598: dimension 1024, modulus $q = 998244353$, secret uniform in $\mathbb{Z}_q$, error uniform on $[-7616, 7616]$ (the centred rounding width $q/(2p)$, standard deviation ≈ 4397), and $m = 2^{20}$ samples, which is one ring element per block of a 1 GB file. Raising m to 2^{22} does not change the figure. The cheapest finite attack the estimator returned is BDD at $\approx 2^{319}$ operations; uSVP is $\approx 2^{322}$ and the dual attack is $\approx 2^{334}$. Coded-BKW did not return a finite cost on this instance. The model does not use the cyclotomic ring, so this is a generic-lattice cost of the same dimension and noise, not a reduction from ideal-SVP. It is the reason these parameters are a security point and not only a timing point: every attack the estimator finished is far above 2^{128} .

7.4 End-to-end USS

Table 3 is not a USS comparison: it times encryption engines. USS is the 2023 protocol with a new HVC backend and a measured large-file path. The two comparisons that show the construction’s advantage are Table 5 (against the only published USS) and Table 7 (against rotating the same repository without a stub).

Table 5: The ASIACRYPT 2023 USS protocol with Catalano–Fiore HVC versus this paper. HVC cells are the same-stack one-shot measurements of Table 1. Verification CRS excludes $\{W_j\}$.

	2023 CF-USS	KHVC-USS
Verification CRS, $n = 1024$	50.5 MB	48.3 KB (1070 $\times$)
Operational CRS, $n = 1024$	50.5 MB	96.3 KB (512 $\times$)
ParGen, $n = 1024$	70.2 s	0.38 s (185 $\times$)
CRS, $n = 2^{14}$	12.8 GB [§]	768 KB / ≈ 1.5 MB
KeyUp, 4×1 GB	— (no public impl.)	254 s / 4240 B

[§] $(16384/1024)^2 \cdot 50.5 \text{ MB} = 12.8 \text{ GB}$. The 512 $\times$ ratio is $50,528,400/(49,440 + 49,152)$. FileUp cost depends on k ; see Table 6.

Table 6: Server opening work for a FileUp that touches k slots, $n = 1024$. KHVC $k = 1$ uses the one-hot closed form (Lemma 2); denser k use FK20. CF cells with $k < n$ are $k \cdot 63.13 \text{ s}/1024$, a linear estimate from the measured dense all-open, not a sparse CF implementation.

k	CF-HVC	KHVC	source
1	$\approx 62 \text{ ms}$	194 ms	CF estimated, KHVC one-hot
8	$\approx 0.49 \text{ s}$	1.67 s	CF estimated, KHVC FK20
64	$\approx 3.9 \text{ s}$	1.69 s	CF estimated, KHVC FK20
n	63.1 s	1.67 s	both measured (38 $\times$ here)

Table 7: Key rotation of the *same* repository. KHVC-USS KeyUp is measured with cached a_b NTTs, including the Ring-LWR body scan. Baselines re-key every file body. Algebraic-file rows have no stub and no OF-PTXT.

scheme	$n \times m $	time	sent	OF-PTXT
KHVC-USS	$4 \times 1 \text{ GB}$	254 s	4240 B	yes
AES client re-enc	$4 \times 1 \text{ GB}$	17.9 s [‡]	4.0 GB	no
Nested AES Upd	$4 \times 1 \text{ GB}$	17.5 s [‡]	4.0 GB	no
ReCrypt-KHPRF (EC)	$4 \times 1 \text{ GB}$	55 min [‡]	8.0 GB	no
SHINE0 on bodies	$4 \times 1 \text{ GB}$	37 min [‡]	4.0 GB	no
RISE on bodies	$4 \times 1 \text{ GB}$	4.6 h [‡]	8.0 GB	no
KHVC-USS	$8 \times 100 \text{ MB}$	58.9 s	4240 B	yes
AES client re-enc	$8 \times 100 \text{ MB}$	3.5 s [‡]	800 MB	no
KHVC-USS	$64 \times 1 \text{ MB}$	2.22 s	4240 B	yes
AES client re-enc	$64 \times 1 \text{ MB}$	0.28 s	64 MB	no

[‡] Not a timed run. $T = T_{1\text{MB}} \cdot n \cdot |m|/1 \text{ MB}$ from **fairbench** medians (AES: client re-encrypt; others: server Upd). Block-wise algebraic throughput is essentially flat in $|m|$; a 1 GB working set may still see cache and memory-bandwidth effects not present at 1 MB.

The durable gain against the 2023 backend is the CRS: at $n = 1024$ the operational parameters are $512\times$ smaller, and at $n = 2^{14}$ Catalano–Fiore is 12.8 GB against about 1.5 MB. One-hot FileUp openings cut the single-slot KHVC cost from 1.62 s to 194 ms at $n = 1024$ (about $3\times$ Catalano–Fiore’s estimated 62 ms, down from a $26\times$ gap when FK20 was used for $k = 1$). Dense FK20 is still about 1.67 s independent of $k \geq 8$, while the dense CF estimate scales as about $61.7k$ ms, so the crossover remains near $k^* \approx 27$. The dense case $k = n$ is where KHVC is about $38\times$ faster. That is why a CF-USS 1 GB experiment is missing: the 2023 CRS does not scale to the n where USS is interesting.

Against payload-only rotation of the same four 1 GB files, KHVC-USS KeyUp is measured at 254 s and 4240 B, including the Ring-LWR body scan with cached a_b . AES re-encrypt is 17.9 s and 4 GB on the wire; SHINE0 or RISE on the file bytes is 37 minutes to 4.6 hours. Those algebraic rows are extrapolations, not timed runs. The USS row is slower than client AES re-encrypt on this machine, but the token is $10^6\times$ smaller than rewriting the bodies, the server does the work, and OF-PTXT is kept.

Table 8: KHVC-USS microbenchmarks (Ring-LWR bodies, cached a_b). FileUp wall-clock includes decrypting file i , encrypting the replacement, and the one-hot open of the delta; KeyUp includes rewriting every body.

n	$ m_i $	ParGen	Store	Rev	FileUp	KeyUp
8	4 KB	3.9 ms	8.0 ms	1.9 ms	4.8 ms	7.5 ms
32	4 KB	7.6 ms	34.5 ms	2.0 ms	19.5 ms	26.3 ms
64	1 MB	16.5 ms	6.08 s	38 ms	0.15 s	2.22 s
8	100 MB	1.6 ms	80.9 s	7.82 s	14.4 s	58.9 s
4	1 GB	1.0 ms	425 s	79.7 s	192 s	254 s

Store, Rev, FileUp, and KeyUp all scale with total payload bytes under the Ring-LWR DEM. KeyUp of four 1 GB files is 254 s: the DEM scan dominates, and the token is 4240 bytes independent of n and of $|m|$. Caching a_b removes the previous rebuild tax (404 s end-to-end with on-the-fly NTT). The extra integrity work for a one-file FileUp is the one-hot open of Lemma 2, not a scan of other plaintexts.

Honest costs. Our FK20 uses projective $\mathbb{G}_1$ arithmetic without a dedicated Pippenger MSM for the Toeplitz step; at $n < 1024$ it loses to the naïve loop in Table 1. The crate now switches there. The CSV behind that table is older than the switch. A production port (blst + FK20 as in Ethereum *c-kzg-4844*) would improve the $n = 2^{14}$ all-open. We did not implement a full CF-HVC USS repository at 1 GB: Table 5 compares the HVC backend that 2023 actually uses, which is the bottleneck.

8 Related work

Vector commitments [2, 11, 12] and polynomial commitments [3] are standard. Fleischhacker–Hall–Andersen–Simkin [14] use hidden KZG as a vector commitment and as a handle for extractable witness encryption. Their interface is a (hiding) polynomial commitment / VC, not an HVC: they do not provide the USS aux-independent-of- n contract, **ComHom**/**OpenHom**, or the position-hiding game with openings of the other slots as group elements. Using a polynomial commitment as a VC is common in proof systems, including Lasso [15]. Those uses likewise omit homomorphism plus short aux, which is what FileUp/KeyUp need.

Transparent FRI commitments [16] avoid a pairing SRS. A FRI opening is not a $\mathbb{G}_1$ element, so it does not plug into the RISE homomorphism used here. That is why this USS instantiation uses KZG.

UE: BLMR, RISE [4], ReCrypt [5], SHINE [6], nested AES and an RLWE KH-PRF [7]. That paper already notes that a Ring-LWR almost key-homomorphic PRF exists and moves to RLWE to widen the noise budget; with a 128-update cap the Ring-LWR pad is enough, and Section 7 measures it. USS [10] is the integrity-aware combination and already states that the DEM may be any IND-CPA UE; this paper is its first linear-SRS instantiation with a measured large-file DEM. Cryptoperiods are NIST’s [9], not a new UE definition.

9 Conclusion

KHVC removes the quadratic CRS that blocked the 2023 USS at realistic n (operational parameters $512\times$ smaller at $n = 1024$). Position hiding is statistical, up to an n/p setup event. Body confidentiality and body update are Ring-LWR in the random oracle: IND-CPA for a fresh body and for a body after at most 127 updates. OF-PTXT does not need the DEM to be authenticated, and it holds if the DEM secret leaks. One-hot FileUp openings bring a single-slot update to 194 ms at $n = 1024$ (about $3\times$ the Catalano–Fiore sparse estimate). On four 1 GB files end-to-end KeyUp is 254 s with a 4240-byte token and cached a_b NTTs. Generic-lattice estimation of this Ring-LWR point puts the cheapest finished attack near 2^{319} , above the 2^{128} line, without using the ring.

References

- [1] D. Boneh, X. Boyen. Short signatures without random oracles. EUROCRYPT 2004.
- [2] D. Catalano, D. Fiore. Vector commitments and their applications. PKC 2013.
- [3] A. Kate, G.M. Zaverucha, I. Goldberg. Constant-size commitments to polynomials and their applications. ASIACRYPT 2010.
- [4] A. Lehmann, B. Tackmann. Updatable encryption with post-compromise security. EUROCRYPT 2018.
- [5] A. Everspaugh, K. Paterson, T. Ristenpart, S. Scott. Key rotation for authenticated encryption. CRYPTO 2017. Implementation: <https://github.com/samscott89/ReCrypt>.
- [6] C. Boyd, G.T. Davies, K. Gjøsteen, Y. Jiang. Fast and secure updatable encryption. CRYPTO 2020, ePrint 2019/1457. Implementation: <https://github.com/YaoJGalteland/SHINE>.
- [7] D. Boneh, S. Eskandarian, S. Kim, M. Fisch. Improving speed and security in updatable encryption schemes. ASIACRYPT 2020, ePrint 2020/222. Implementation: https://github.com/moshih/UpdateableEncryption_Code.
- [8] M.R. Albrecht, R. Player, S. Scott. On the concrete hardness of Learning with Errors. Journal of Mathematical Cryptology 9(3), 2015. Code: <https://github.com/malb/lattice-estimator>.
- [9] National Institute of Standards and Technology. Recommendation for key management: Part 1 — general. NIST Special Publication 800-57 Part 1, Revision 5, 2020. Table 1: symmetric data-encryption keys, originator-usage period < 2 years.
- [10] L. Chen, H. Guo, Y.-N. Li, Q. Tang. Efficient secure storage with version control and key rotation. ASIACRYPT 2023, ePrint 2023/1421.
- [11] M. Campanelli, D. Fiore, N. Greco, D. Kolonelos, L. Nizzardo. Incrementally aggregatable vector commitments. ASIACRYPT 2020.

- [12] D. Boneh, B. Bünz, B. Fisch. Batching techniques for accumulators. CRYPTO 2019.
- [13] D. Feist, D. Khovratovich. Fast amortized KZG proofs. ePrint 2023/033 (FK20/FK22).
- [14] N. Fleischhacker, M. Hall-Andersen, M. Simkin. Extractable witness encryption for KZG commitments. ASIACRYPT 2024.
- [15] S. Setty, J. Thaler, R. Wahby. Unlocking the lookup singularity with Lasso. EUROCRYPT 2024.
- [16] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. ICALP 2018.
- [17] <https://github.com/YaoJGalteland/SHINE>
- [18] <https://github.com/samscott89/ReCrypt>
- [19] https://github.com/moshih/UpdateableEncryption_Code